StockFlow

An inventory app, shipped the DevOps way

Tests → Docker → CI/CD + Trivy → GHCR → Terraform (AWS EKS) → Helm → Prometheus + Grafana

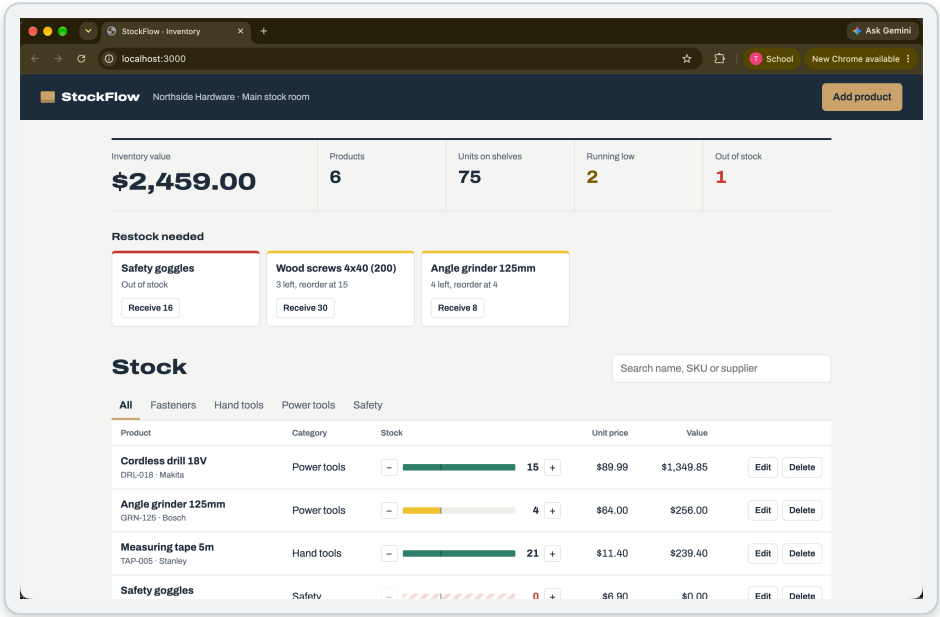
github.com/tirth-alt/stockflow-devops

The app: a stock room that flags what to reorder

1. **Products CRUD**
   Name, SKU, category, quantity, reorder level, price, supplier

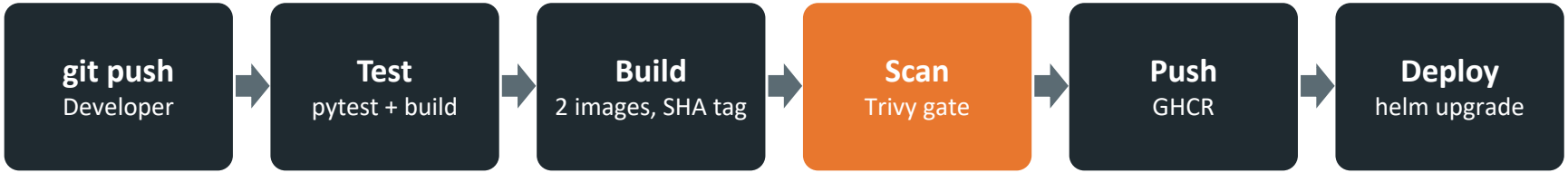
2. **Restock queue**
   Low stock (qty ≤ reorder level) and out of stock, at the top of the page

3. **Inventory value**
   Totals from /api/products/stats



React → Nginx → FastAPI → PostgreSQL

Architecture: git push to a monitored cluster



Kubernetes · namespace stockflow · installed with Helm

Ingress (nginx)
stockflow.local

frontend ×2
Nginx + React · /

backend 2–6
FastAPI · /api · HPA 60% CPU

postgres
PostgreSQL 16 + PVC

Terraform → AWS

VPC (2 public + 2 private subnets, NAT) + EKS 1.36 + managed node group

Observability

Prometheus scrapes /metrics via a ServiceMonitor; Grafana shows the StockFlow API dashboard

Tech stack

Frontend
React 19 + Vite, served by non-root Nginx

Backend
FastAPI, SQLAlchemy 2, Pydantic

Database
PostgreSQL 16, Alembic migrations

Tests
pytest, in-memory SQLite (11 passing)

Containers
Dockerfiles + Docker Compose

CI/CD
GitHub Actions

Security
Trivy, HIGH/CRITICAL gate

Registry
GHCR, commit-SHA tags

Infrastructure
Terraform: AWS VPC + EKS

Deployment
Helm chart, Ingress, HPA

Monitoring
kube-prometheus-stack

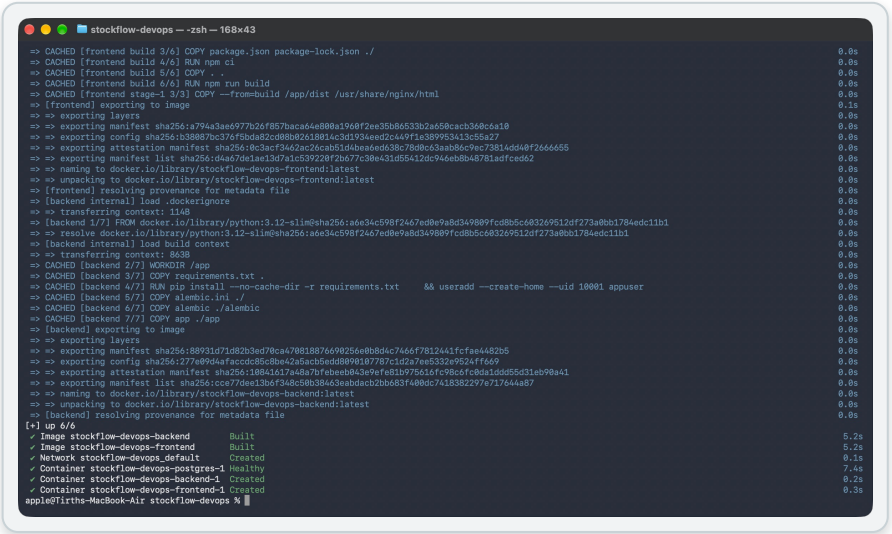
Troubleshooting
Deliberately broken manifests

Containers: small, non-root, reproducible

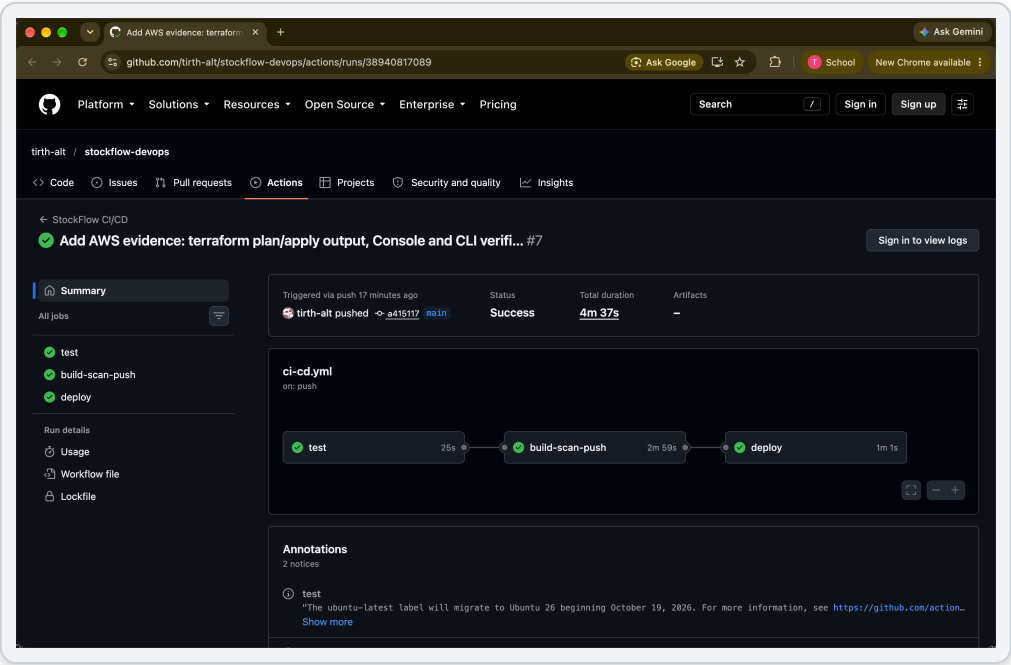
1. **Non-root by default**
   Backend runs as UID 10001; frontend uses nginx-unprivileged (UID 101, port 8080)

2. **Multi-stage frontend**
   Node build stage → Nginx runtime stage, about 80 MB

3. **Safe migrations**
   Alembic runs at start behind a Postgres advisory lock, so replicas don't race



CI/CD: every push to main ships itself



test pytest + frontend build; a failure stops everything

build-scan-push Docker build, Trivy, push to GHCR

deploy helm upgrade --install on a self-hosted runner

Images tagged with the commit SHA, never latest

Security gate: Trivy blocks known CVEs

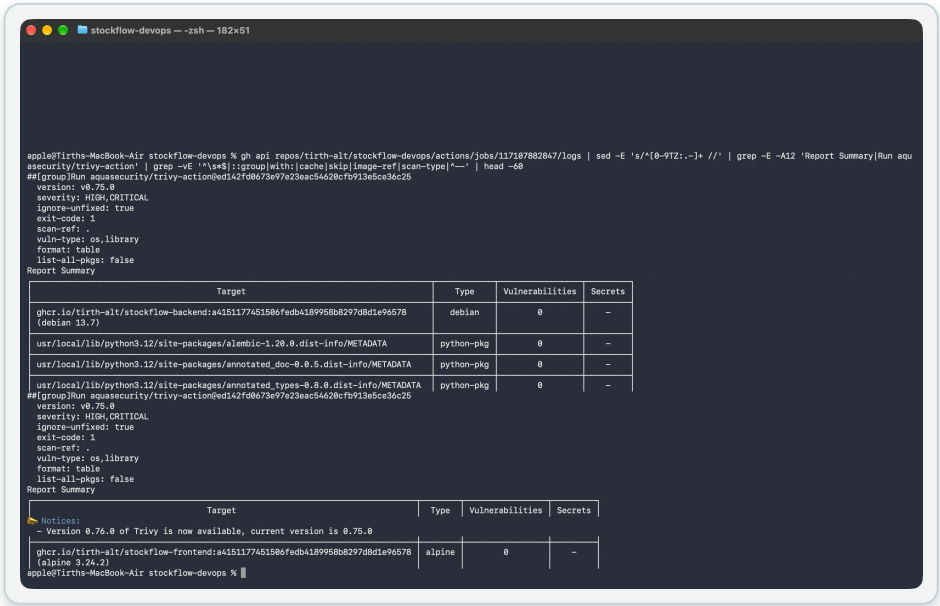
0

HIGH/CRITICAL fixable CVEs in both images today

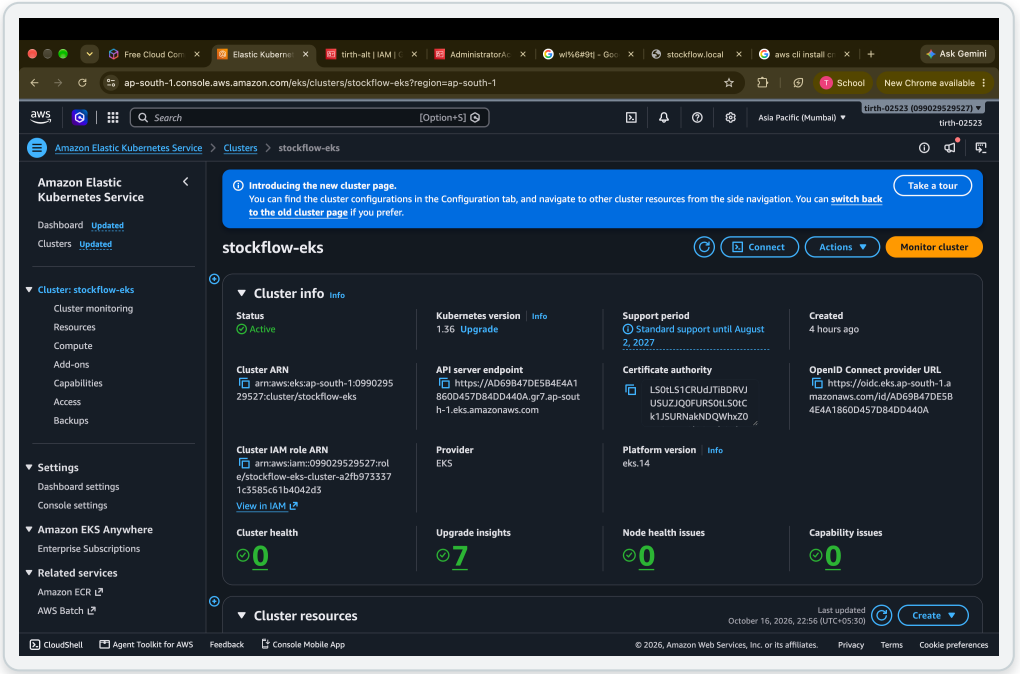
The gate caught a real one

The first scan failed the frontend: the nginx 1.29 base shipped a vulnerable pcre2 and libtiff. Moving to nginx 1.30 brought patched packages and the gate turned green.

A clean scan is not proof of safety: it doesn't cover our own code, leaked secrets or future CVEs



Terraform: AWS VPC + EKS as code



1.36

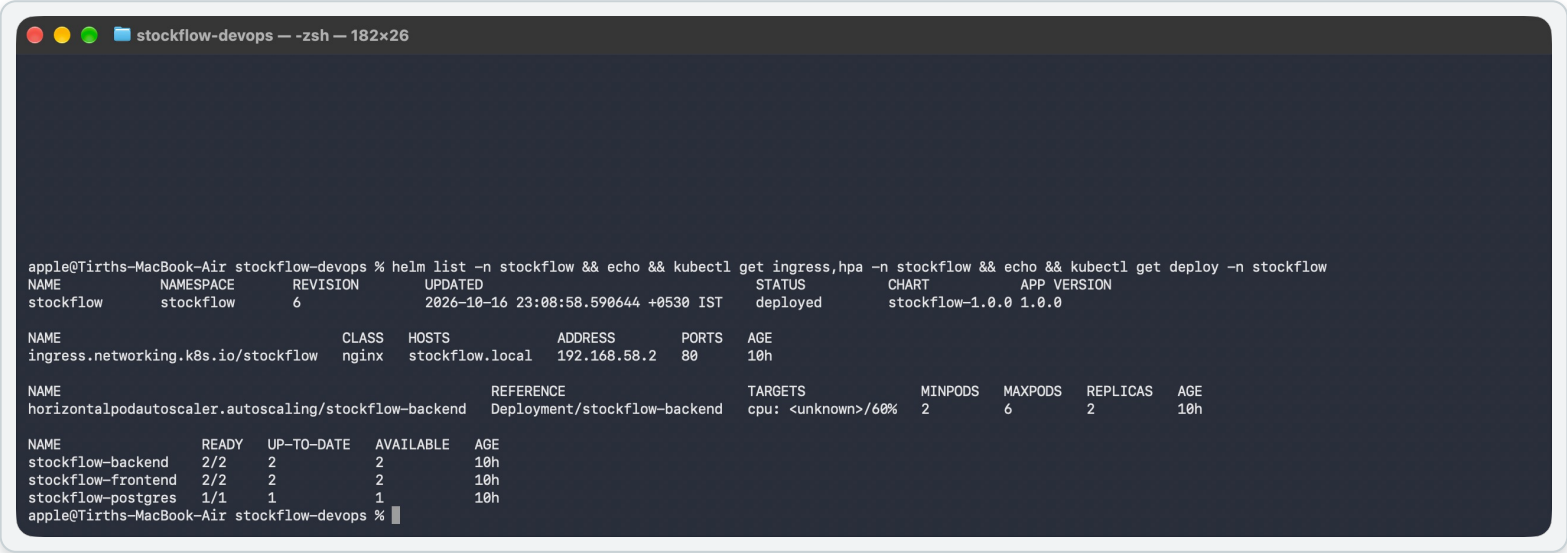
EKS version, in standard support ($0.10/h control plane, not $0.60)

$0.25/h

for the whole stack, so terraform destroy runs at the end

2 public + 2 private subnets across 2 AZs, NAT, c7i-flex.large nodes (free-tier eligible)

Kubernetes with Helm



1 **Ingress**
One hostname; / → frontend, /api → backend

2 **Probes**
/health = alive (no DB); /ready = DB answers

3 **No latest**
The chart refuses to render without an image tag

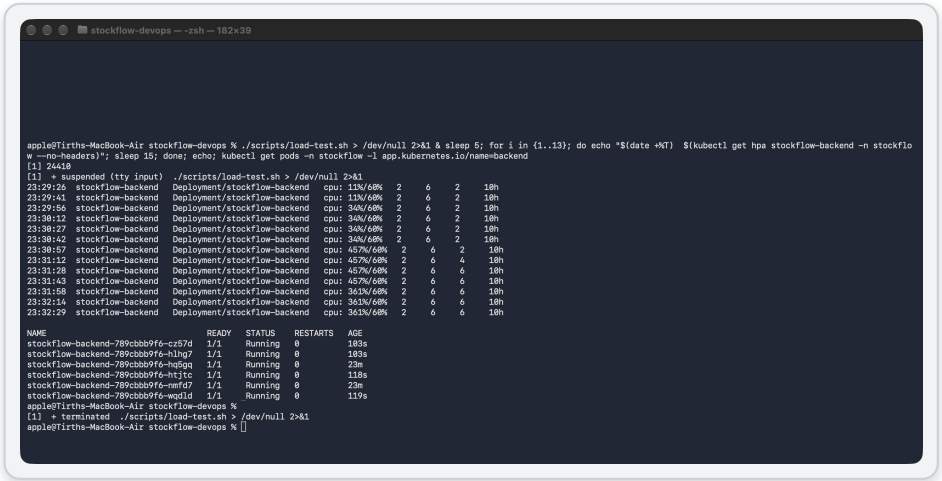
Autoscaling under load

11% → 457%

backend CPU against a 60% target during scripts/load-test.sh

2 → 6 pods

HPA scaled the backend 2 → 4 → 6, then back down



Observability: Prometheus + Grafana



- Requests per second
- p95 latency
- 5xx error rate
- Running pods and CPU

The dashboard ships inside the Helm chart, and a ServiceMonitor tells Prometheus to scrape /metrics

Troubleshooting labs: break it, then fix it

ImagePullBackOff

Cause

Tag v9.9.9-does-not-exist was never pushed

How I found it

kubectl describe pod → Events: manifest unknown

Fix

Point the Deployment at a real commit-SHA tag

Service with no endpoints

Cause

Selector says backnd, pods say backend

How I found it

kubectl get endpoints → <none>

Fix

Patch the selector; endpoints list pod IPs again

Thank you

Code, docs and evidence: github.com/tirth-alt/stockflow-devops

Demo video: demo/stockflow-demo.mp4